EXPERIMENT NO. 6

Sort a List and Find the Maximum Element

Programming Language: Java

Name : A Yuvan Charan
Register No. : 192424383

Aim

To write a Java program that first sorts a list using an efficient sorting algorithm (merge sort) and then finds the maximum element of the sorted list.

Question

You have an algorithm that processes a list of numbers. It first sorts the list using an efficient sorting algorithm and then finds the maximum element in the sorted list. Write the code for the same.

Test Case 1 - Empty List

Input: []

Expected Output: None or an appropriate message indicating that the list is empty.

Test Case 2 - Single Element List

Input: [5]

Expected Output: 5

Test Case 3 - All Elements are the Same

Input: [3, 3, 3, 3, 3]

Expected Output: 3

Algorithm

Step 1: Start.

Step 2: Read the list and its size n. If n = 0, print that the list is empty and stop.

Step 3: Sort the list using merge sort: split it into two halves, sort each half recursively, and merge the sorted halves.

Step 4: After sorting, the maximum element is the last element, arr[n-1].

Step 5: Print arr[n-1].

Step 6: Stop.

Program (Java)

```
public class Main {

    static void merge(int[] a, int l, int m, int r) {
        int[] tmp = new int[r - l + 1];
        int i = l, j = m + 1, k = 0;
```

```
        while (i <= m && j <= r)
            tmp[k++] = (a[i] <= a[j]) ? a[i++] : a[j++];
        while (i <= m) tmp[k++] = a[i++];
        while (j <= r) tmp[k++] = a[j++];
        for (k = 0; k < tmp.length; k++)
            a[l + k] = tmp[k];
    }

    static void mergeSort(int[] a, int l, int r) {
        if (l >= r) return;
        int m = (l + r) / 2;
        mergeSort(a, l, m);
        mergeSort(a, m + 1, r);
        merge(a, l, m, r);
    }

    static void sortAndFindMax(int[] a) {
        if (a.length == 0) {
            System.out.println("Output: None (the list is empty)");
            return;
        }
        mergeSort(a, 0, a.length - 1);
        System.out.println("Output: " + a[a.length - 1]);
    }

    public static void main(String[] args) {
        sortAndFindMax(new int[] {});                  // Test Case 1
        sortAndFindMax(new int[] {5});                 // Test Case 2
        sortAndFindMax(new int[] {3, 3, 3, 3, 3});     // Test Case 3
    }
}
```

Output

```
Output: None (the list is empty)
Output: 5
Output: 3

=== Code Execution Successful ===
```

Complexity Analysis

Time Complexity: O(n log n) - merge sort dominates; finding the maximum of the sorted list is O(1).
Space Complexity: O(n) - merge sort uses a temporary array.

Result

Thus the program was executed successfully and the output was verified.